

Applied Prompt Patterns, Advanced Reasoning & Production Practices

5 Applied Prompt Patterns

1. Extraction Tasks

Description: Pull out specific pieces of information from unstructured or semi-structured text and organize them into a structured format.

Key Uses:

- Extract entities (people, organizations, locations, products)
- Identify key facts, events, amounts, and relationships
- Capture dates, deadlines, and action items

Best Practices:

- Always specify the exact fields you want to extract.
- Request structured output (JSON or Markdown table) for easy parsing.
- Mention what to do with missing information.

Practical Example:

- **Input:** "Project meeting with Acme Corp. and team leads, set for next Tuesday, November 12th at 2PM, focused on Q4 budget. Sarah Chen attending."
- **Good Prompt Instruction:** "Extract the following information in JSON format: project name, client, date, time, attendees."
- **Output:** Clean structured data ready for database or downstream processing.

Use Cases: Invoice processing, resume parsing, contract analysis, meeting notes.

2. Classification Tasks

Description: Assign input text into one or more predefined categories or labels.

Key Uses:

- **Intent Detection:** Understand what the user wants to do.
- **Sentiment Analysis:** Positive, Negative, Neutral, or more nuanced emotions.
- **Routing Decisions:** Route queries to the right team or department.

Practical Examples:

- **Sentiment Analysis:**
 - Input: "The new app update is fantastic! So much faster."
 - Output: [POSITIVE] with confidence score (optional).
- **Intent Detection:**
 - Input: "How do I reset my password?"
 - Output: [ACCOUNT_SUPPORT]

Tip: Provide clear category definitions and few-shot examples for better accuracy.

Use Cases: Customer support automation, content moderation, email triage, lead qualification.

3. Transformation Tasks

Description: Take existing content and change its form, length, language, or style while preserving the original meaning.

Common Types:

- Summarization (long → short)
- Translation (language to language)
- Reformatting (paragraph → bullet points / table)
- Style Transfer (casual → professional / formal → friendly)

Practical Examples:

- **Summarization:** Convert a 500-word meeting transcript into a 100-word executive summary with key decisions and action items.
- **Translation + Style:** Translate technical documentation into simple language for non-technical users.

Best Practice: Clearly define target length, tone, and audience in the prompt.

4. Generation Tasks

Description: Create new original content based on instructions, context, and constraints.

Common Uses:

- Writing reports, emails, and proposals
- Code generation (Python, SQL, JavaScript, etc.)
- Creative content (stories, poems, marketing copy, social media posts)

Practical Examples:

- **SQL Generation:** "Get all customer emails from New York who made a purchase last month" → Accurate SQL query.
- **Creative Generation:** "Write a short motivational LinkedIn post about overcoming failure."

Critical Success Factor: Use COSTAR framework + clear constraints (tone, length, style, format) for high-quality generation.

5. Decomposition

Description: Break a large, complex task into smaller, sequential, or parallel sub-tasks.

Why It's Powerful for Agents:

- Makes difficult problems solvable
- Improves transparency and debuggability
- Reduces hallucinations
- Enables better error handling and recovery

Example Flow (Market Research Agent):

1. Define research objectives and keywords
2. Search for latest information
3. Extract and summarize key findings from each source
4. Analyze trends and competitors
5. Synthesize final insights and recommendations

Tip: Use Chain of Thought together with Decomposition for best results.

Advanced Reasoning Techniques

Chain of Thought (CoT) Prompting

- **Core Idea:** Explicitly ask the model to think step-by-step before giving the final answer.
 - **Magic Phrase:** "Think step by step", "Let's think through this logically", or "Explain your reasoning".
 - **Benefit:** Significantly boosts performance on arithmetic, logic, commonsense reasoning, and multi-step problems.
 - **When to Use:** Almost any reasoning task.
-

Self-Consistency Technique

- **How it Works:** Run the same prompt multiple times (3–5 times) with temperature > 0, then select the most consistent answer (majority vote).
 - **Benefit:** Reduces randomness and hallucinations.
 - **Best For:** Factual questions, math, logical reasoning, and decision-making.
-

Self-Refine / Critique & Improve

Iterative Process:

1. **Generate** — Create initial response.
2. **Critique** — Ask the model: "Review your answer. What are the weaknesses? How can it be improved?"
3. **Refine** — Generate an improved version based on the critique.

Benefit: Produces thoughtful, high-quality, polished outputs.

Best For: Writing, analysis, strategic recommendations.

Least-to-Most Prompting

- **Strategy:** Solve the easiest sub-problems first, then gradually tackle harder ones using previous solutions.
 - **Advantage:** Builds confidence and accuracy progressively.
 - **Ideal For:** Complex word problems, multi-step planning, and mathematical reasoning.
-

Tree of Thoughts (ToT)

- **Overview:** The model explores multiple different reasoning paths (branches), evaluates each one, and selects or combines the best paths.
 - **When to Use:** Strategic planning, complex problem-solving, coding architecture, or research.
 - **Trade-off:** More computationally expensive but can deliver superior results.
-

Prompt Management & Production Realities

Prompt Versioning Strategies

- **Code-based Versioning** (Recommended for developers): Store prompts in `.py` files or YAML/JSON with Git versioning.
- **Managed Platform Versioning**: Use tools that offer visual diff, collaboration, and easy deployment.

Best Practice: Maintain prompt history and track performance of each version.

A/B Testing Prompt Variants

Structured Process:

1. Define clear success metrics (accuracy, user satisfaction, conversion rate, etc.)
2. Create controlled variations of prompts
3. Run experiments on live or test traffic
4. Analyze statistical significance and roll out the winner

This is the scientific way to improve prompts continuously.

Prompt Caching and Cost Optimization

Major Benefits of Caching:

- Reduced latency (faster responses)
- Lower token usage and cost
- More consistent outputs

Other Cost Optimization Techniques:

- Token minimization (be concise)
 - Use smaller/cheaper models for simple tasks
 - Semantic caching for similar queries
 - Prompt standardization across the team
-

Overall Best Practices from Session 2

- Combine multiple techniques (e.g., Decomposition + Chain of Thought + Structured Output).
 - Always iterate and refine prompts — never settle for the first version.
 - Measure prompt performance with clear metrics.
 - Think like an engineer: version, test, monitor, and optimize.
-

End of Session 2 Detailed Notes

Revision Tips:

- Practice **Decomposition + Chain of Thought** daily — this combination is extremely powerful.
 - Try converting simple tasks into the 5 Applied Patterns.
 - Experiment with Self-Refine on important outputs.
-